

AWS Interview Questions & Answers

1) Can we change the public EC2 instance into a private instance without touching its route table?

Answer:
Concept: Think what makes an ec2 public? Its public ip and public subnet having igw in route table, nacl.

ENI CONCEPT.

- Stop ec2 instance and public IP will be released if EIP is not associated but another public ip will be reassigned when we restart.
- To avoid reassignment of public ip, Go to Networking of Instance, Find it's ENI.
- Go to ENI. In its setting find manage ip address→ disable auto assign public ip.
- Now when you restart your EC2 , it will not have public IP address.
- Note: In reality, Public and Private IP are allocated to Elastic Network Interface(ENI) and ENI is associated with instances.

2) How to allow EC2 A (in VPC A) to talk to EC2 B (in VPC B), but block EC2 A from talking to EC2 C (in same VPC B)?

Answer: Use VPC Peering between VPC A and B. Then configure Security Groups so A can reach B but not C. Apply strict Inbound Rules on EC2 C's security group to deny traffic from A's IP/subnet. Avoid using broad rules that apply to entire VPC.

VPC	CIDR Block	EC2 Instance	IP
VPC A	10.10.0/16	EC2 A	10.10.1.10
VPC B	10.20.0/16	EC2 B	10.20.1.20
		EC2 C	10.20.1.30



2



3



4



5



6



7



8



9

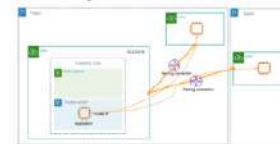


Update route tables so that VPC A and VPC B can reach each other only for specific ranges.

Only route to EC2 B's IP, not entire VPC B (10.20.0.0/16) – this limits traffic.

Component	Action
VPC Peering	Create peering between VPC A and B
Routing	In VPC A, only route to 39.26.3.26/32 (not full VPC B range)
Security Group EC2 B	Allow traffic from 39.26.3.26/32
Security Group EC2 C	No rule allowing traffic from EC2 A
Optional	Restrict EC2 A outbound traffic only to EC2 B's IP

VPC Peering



3) How do you provide static public IP of your choice like 3.10.71.104 to AWS EC2 instance? OR Public IP of your server getting change every time it reboots. How shall you make some permanent IP assigned to it?

Answer:

Static public gives hint for EIP. We cannot choose public IP of our choice.

No, you cannot create an Elastic IP (EIP) of your choice in AWS

Concept:

Elastic IPs are static public IPv4 addresses that AWS allocates from a pool owned by AWS.

Use Elastic IP (EIP) to assign a static public IP to an instance. Go to EC2 dashboard → Elastic IPs → Allocate a new IP → Associate with your instance

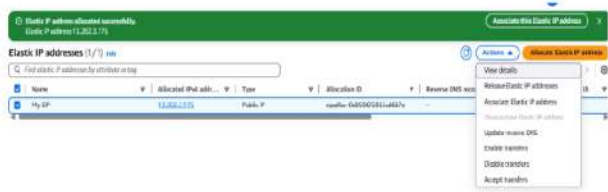


Search



ENG IN 12:14 17-05-2026

It **doesn't change** [static] when the instance stops or restarts. You must explicitly associate it to your EC2, and remember—EIP is chargeable if not attached to a running instance. You must release it when not in use.



Name	Allocated IP address	Type	Allocation ID	Reverse DNS
My-IP	12.34.56.78	Public IP	ea9fbc-6400000000000000	

4) Suppose you have an e-commerce application deployed on AWS. How will you handle high traffic spike during some festivities or SALE period?

Answer:
CONCEPT: ASG + LB COMBO

Use Auto Scaling Groups with proper dynamic scaling policies.
ASG will increase the ec2 when needed.
Load balancer will distribute the traffic between the ec2 machines.
Use CloudFront CDN to cache content globally and reduce backend load.
Monitor traffic with CloudWatch and scale database using RDS read replicas or Aurora.

5) Can you tell what is the default route in main route table? What does it actually mean?

Answer:

If from ec2 A you want to ssh to other ec2 B within same vpc, this request will have destination=IP address of ec2 B [which belongs to vpc cidr range] , so route this request locally inside vpc. No need to go outside of vpc.



If destination=VPC CIDR [address of any resource inside vpc] , then send it to Target=local[means keep it inside vpc]
All instances **within the same VPC** can communicate with each other **by default**.

On Ec2, we want to do take git pull. This request will be for the IP address of the git repository which is on the internet [i.e. outside this vpc], the request is sent to IGW. If destination=0.0.0.0/0 [If an EC2 tries to reach any IP address that's not in the VPC CIDR range, the address belongs to outside vpc], then the request is sent to the internet [target=internet gateway].
If the traffic is meant for **any IP outside the VPC**, forward it to the **Internet Gateway**.

Note: If the destination Ip does not match with any rule in table, then it goes to rule with destination=0.0.0.0/0
It is like a **default path** — a backup rule that catches all remaining traffic not matched by earlier routes.

rtb-0e66f0d19fd16750d

Details Routes Subnet associations Edge associations Route propagation Tags

Routes (2) Main route table having igw entry and local entry for vpc CIDR

Destination	Target	Status	Propagated
0.0.0.0/0	igw-0MG7b1c3015d8f907	Active	No
172.31.0.0/16	local	Active	No

6) Can you EC2 to S3 without Internet? Application is hosted on ec2 and it has to fetch some objects stored on s3 bucket. How shall you do it without internet connectivity?

Answer:

- Use a VPC Endpoint (Gateway Type) for S3.
- It creates a **private connection between your VPC and S3**, bypassing the need for an Internet Gateway or NAT Gateway.
- Ensure the EC2 has appropriate **IAM** role attached.

Ensure S3 bucket policy allows VPC endpoint access.

Concept:

Page 5 / 158

3

4

5

6

7

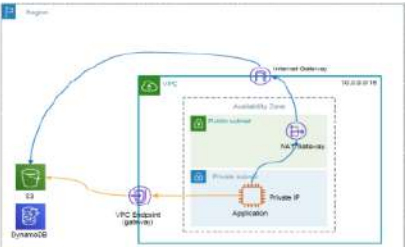
8

9

10

VPC Endpoints enable private connections from your VPC to supported AWS services (like S3, DynamoDB) without needing internet access, NAT, or gateway. It enhances security and reduces data transfer costs.

VPC Endpoints & Privatelink



1 CREATE A VPC ENDPOINT IN VPC

Endpoints (1/1) info

Name	VPC endpoint ID	Endpoint type
demo-endpoint	vpc-e7b8417ded6d1e2	Gateway

2. IN PRIVATE ROUTE TABLE, ADD ENTRY FOR VPC-ENDPOINT

rtb-014410368da216a5f / private-route

Details

Route table ID: rtb-014410368da216a5f

Main: No

Explicit subnet associations: subnet-0176a5d209f71701 / private-subnet

VPC: vpc-c0f6a6d10a8f9a55d / demo-vpc

Client ID: 17142095731

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

In Route table add entry for vpc endpoint.

Destination	Target	Status
0176a5d209f71701	vpc-c0f6a6d10a8f9a55d	Active
10.0.0.0/16	local	Active

7) How do we ensure EC2 can access the internet, but external traffic can't reach it? You have to make your ec2 secure that from outside internet, traffic cannot access it.

Answer:

- * Launch EC2 in a private subnet.
- * Attach a NAT Gateway in a public subnet.
- * Add the entry for NAT gateway in route table for the private subnet.
- * Do not associate public IP or allow inbound traffic in the security group.

PRIVATE ROUTE TABLE HAS ENTRY FOR NAT

Page 7 / 158

75%

Routes (5)

Destination	Target	Status
0.0.0.0/0	nat-0b0548eac9-51075d	Active
10.0.0.0/16	local	Active

8) How will you make an application deployed on EC2 highly available and scalable?

Answer:

CONCEPT: **highly available** = MULTI AZS WHILE CREATING LB + **scalable**= ASG

Deploy the app in an Auto Scaling Group across multiple Availability Zones. Use ALB to distribute traffic. Store static content in S3 and use CloudFront. Monitor with CloudWatch.

9) What are the different scaling policies that we can implement with ASG.

Answer: An Auto Scaling Group (ASG) automatically scales EC2 instances based on demand. There are 3 kinds of policies: Dynamic, predictive, Scheduled action

Policies include:

- **Target tracking scaling**—Increase and decrease the current capacity of the group based on a Amazon CloudWatch metric and a target value. It works similar to the way that your thermostat maintains the temperature of your home—you select a temperature, and the thermostat does the rest.

Example: If you set a target CPU utilization of 50%, Auto Scaling adds or removes instances as needed to keep CPU usage around 50%.

- **Step scaling**—Step scaling adjusts the capacity of the Auto Scaling group based on the magnitude of the alarm breach. Larger breaches lead to larger scaling adjustments.

Example: If the CPU utilization exceeds 70%, you can configure the policy to add 2 instances for a small breach (e.g., CPU at 75%) and add 5 instances for a larger breach (e.g., CPU at 90%).

- **Simple scaling**—Increase and decrease the current capacity of the group based on a single scaling adjustment, with a cooldown period between each scaling activity.

Example: If an alarm is triggered when CPU utilization goes above 70%, Auto Scaling adds 2 instances and waits for a specified cooldown time (e.g., 5 minutes) before allowing another scaling action.

Thumbnail 5

Thumbnail 6

Thumbnail 7

Thumbnail 8

Thumbnail 9

Thumbnail 10

Thumbnail 11

Create dynamic scaling policy

Policy type

- Target tracking scaling
- Target tracking scaling ✓
- Step scaling
- Simple scaling

Metric type info
Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization

Target value

50

Instance warmup info
300 seconds

☐ Disable scale in to create only a scale-out policy

Create dynamic scaling policy

Policy type

Target tracking scaling

Scaling policy name

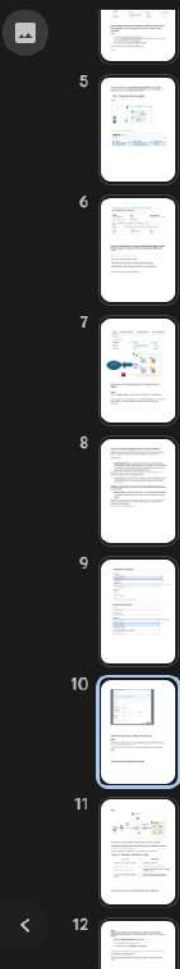
Target Tracking Policy

Metric type info
Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization

Search metric types

- Average CPU utilization ✓
- Average network in (bytes)
- Average network out (bytes)
- Application Load Balancer request count per target
- Custom CloudWatch metric



Create scheduled action

Name:

☒ Provide at least one value for Desired, Min, or Max Capacity

Desired capacity	Min	Max
10	5	12

Recurrence:

Time zone:

Specific start time:

End by:

[Learn more about scheduled actions](#)

10) What is the default behaviour of NACL and Security Group?

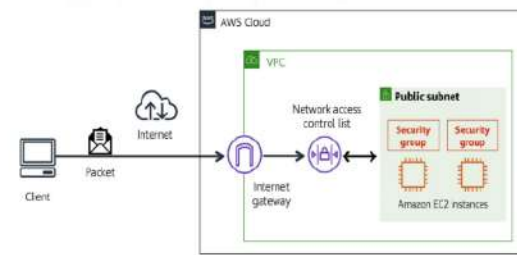
Answer:

- * Security Group: Denies all inbound by default, allows all outbound. We have to add rules to allow incoming traffic at ports.
- * NACL: Allows all traffic by default unless rules are changed. First matching rule is applied.

11) When should we use NACL and Security Group?



Answer:



- * Security Groups: Act as virtual firewalls at the instance level. They are stateful.
 - * NACLs (Network ACLs): Control traffic at the subnet level. They are stateless, meaning return traffic must be explicitly allowed.
- Use both for layered security—NACLs for broad rules, SGs for finer control.

Security Groups vs Network ACL

Security Group	Network ACL
Operates at EC2 instance or ENI level	Operates at Subnet level
Supports only Allow rules	Supports both Allow and Deny rules
Stateful – Return traffic is allowed	Stateless – Return traffic needs to be authorized in Outbound rules
All rules are evaluated before making a decision	Rules are evaluated in the order (lower to higher) and first matching rule is applied

12) EC2 instance cannot be reached via SSH. NACL is open. What's next?



6

7

8

9

10

11

12

13

Answer:

CONCEPT: SSH AT PORT 22 ALLOW IN SG RULES. DIRECT SSH INTO THE EC2-> ACCESSING OVER THE INTERNET-> PUBLICLY ACCESS-> IGW - NACL - SG RULES - PUBLIC IP

- Verify SG Inbound rule for port 22 from your IP.
- Ensure **outbound rule** allows response.
- Confirm instance is in **running** state with **public IP**.

* Security Group: Denies all inbound by default, allows all outbound. We have to add rules to allow incoming traffic at ports.

* NACL: Allows all traffic by default unless rules are changed. First matching rule is applied.

13) Suppose you have large, archived data and this needs to be deleted when it gets one year older. What action will you take to store this?

Answer: Store data in S3 with the Glacier or Glacier Deep Archive storage class to save cost.
Create a Lifecycle Rule on the bucket to delete objects that are older than 365 days.



Search



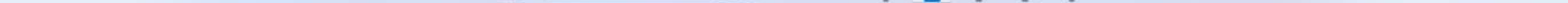
ENG

IN



12:13

17-05-2026





- * Launch EC2 in a public subnet
- * Attach an Internet Gateway to the VPC
- * Update route table to include route to IGW
- * Enable auto-assign public IP
- * Allow required inbound ports in the security group (e.g., 22, 80)

Instance summary for i-99a667b3d9aa50ea1 (test1) [View](#)

[Connect](#) [Instance state](#) [Actions](#) **Has publicip**

Updated less than a minute ago

Instance ID	Public IPv4 address	Private IPv4 addresses
i-99a667b3d9aa50ea1	18.61.219.47 Open address	172.15.2132
IPv6 address	Instance state	Public DNS
	Running	a2-18-61-219-47.ap-southeast-1.amazonaws.com

In route table,

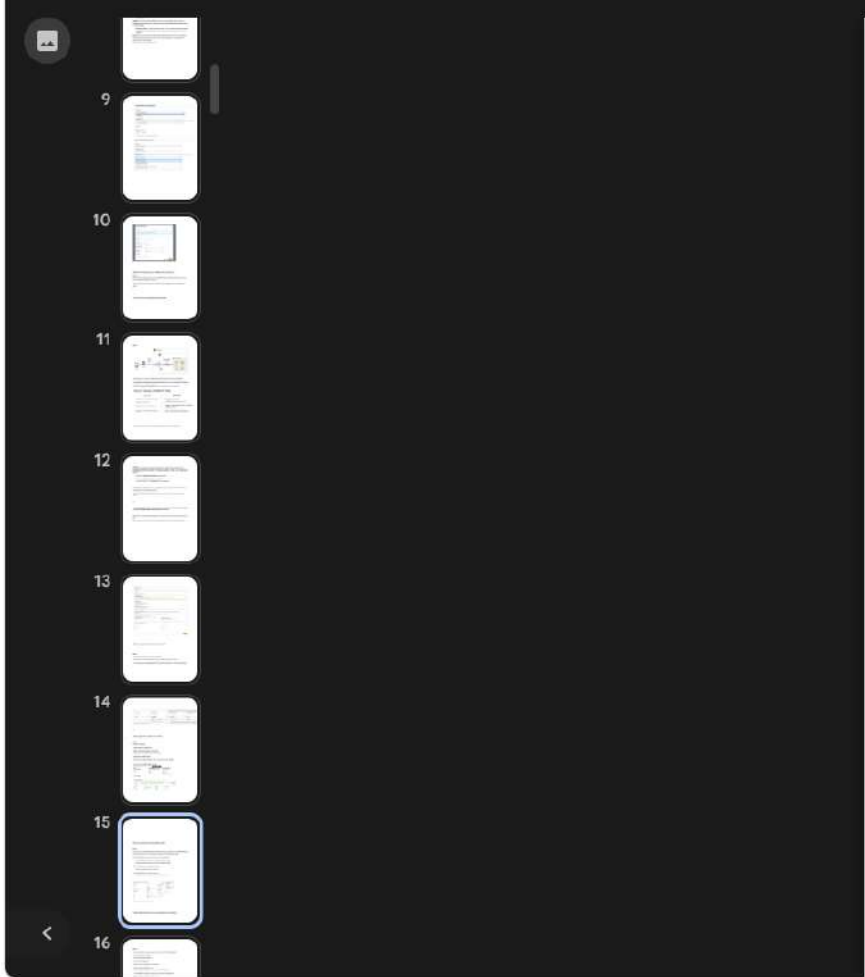
ns-0666d19f16750e

Details Routes Subnet associations Edge associations Route propagation Tags

Routes(2)

Max route table having ipv4 entry and local entry for ip: CIDR

Destination	Target	Status	Accepted
3.0.0.0/8	100.64.0.0/16	Active	No
172.17.0.0/16	local	Active	No



...

16) Can we change the private IP address of EC2?

Answer:
Yes, but only for stopped instances and for secondary private IP, you can attach/detach. After stopping the EC2, you can attach a new one from the subnet's range.

Create a new ENI in the same subnet with a new private IP:

- EC2 → Network Interfaces → Create network interface
- Assign desired private IP from the same subnet range.

Detach old ENI from the stopped instance:

- Select old ENI → Actions → Detach

Attach new ENI to your stopped instance:

- New ENI → Actions → Attach → choose your instance



...

17) What might be the reason for your target group to be unhealthy?



Answer:

- * Incorrect health check path or port. "http://34.107.34.9:3000/health"
- * App not running or crashing
- * Security group blocking traffic
- * NACLs restricting traffic
- * Target instance deregistered or stopped
- * Health check threshold not met.
- * Grace period[30 sec] is less than app bootstrap time[2 min].

Check target group settings, app logs, and network configuration.

Targets Monitoring Health checks Attributes Tags

Registered targets (1) [See](#) [Accuracy-integration test application](#) [Register target](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Amazon distributes its automatically applied to all targets in the target group with at least 1 healthy target.

Instance ID	Port	Zone	Health status	Health status details	Actions
i-0b4t19f8a233a419	3000	us-east-1 (us-east-1a)	Unhealthy	Health check failed	No 0

Targets Monitoring **Health checks** Attributes Tags [See](#)

Health check settings

Protocol	Path	Port	Health threshold
HTTP	/health	3000	2 consecutive health checks successful

Unhealthy threshold	Timeout	Interval	Success ratio
2 consecutive health check failures	30 seconds	30 seconds	20%

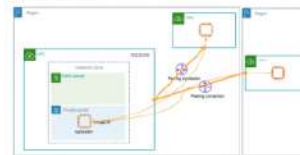
18) Application hosted on ec2 wants to connect to an RDS in another VPC. Is it possible? OR How to establish communication between two VPCs?

Answer:

Use VPC Peering (one-to-one) or Transit Gateway (one-to-many). Configure route tables in both VPCs and ensure security groups allow the required traffic.

- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18

VPC Peering



19) When to use VPC Peering and Transit Gateway?

Answer:

* VPC Peering: For one-to-one connection between 2 VPCs. Simple and cost-effective.

* Transit Gateway: For hub-and-spoke model where multiple VPCs and on-prem are interconnected centrally. Better for scalability.

Note: vpc peering - max 50 per vpc. not transitive - only one to one between 2 vpc. in same or different region possible. [same account or diff aws account also]

traffic flows through private amazon backbone network. [encrypted] not through internet. No igw or nat required for second ec2.

Create vpc peering request- requester - vpc A then give acceptor as vpc B. Then go into region/account where vpc B is present and accept the connection request. Then in Route table of VPC A add rule = If dest=vpc B CIDR then target = VPC peering

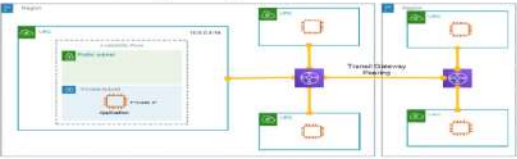
connection. In route table of vpc B, add rule [If dest=VPC A CIDR then target = vpc peering connection]

Allow in security groups also add incoming ssh from vpc cidr of vpc A allowed

Example, spoke model, all customer vpc are connected to a central vpc [where grafana or jenkins is there through vpc peering, but all customer vpc cant talk to each other]

Limitations- not transitive. not possible for overlapping vpc cidr. only 1 peering b/w 2 vpc.

Transit Gateway peering



20) Which AWS service can be used for storing structured data (like rows and columns)?

Answer:

- * Amazon RDS: Managed relational database service [supports MySQL, PostgreSQL, etc.]
- * Amazon DynamoDB: NoSQL, key-value store but also supports structured data

Use RDS if SQL and complex relationships are needed.

21) What are all different types of storage class in S3?

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

13

14

15

16

17

18

19

20

Answer:

- * Standard – frequent access
- * Intelligent-Tiering – automatic cost optimization
- * Standard-IA – infrequent access
- * One Zone-IA – cheaper, single AZ
- * Glacier – archival, low cost, slow access
- * Glacier Deep Archive – lowest cost, longest retrieval time

Used for cost optimization based on access patterns.

...

22) What is Trust Relationship and policy?

Answer:

A trust relationship is a JSON document in IAM roles that defines which entities (users, services, or other roles) are allowed to assume the role. IAM policy defines what actions are allowed or denied on which resources. The trust policy enables access, and permission policies control what can be done once assumed.

...

Select trusted entity

Trusted entity type

- ☒ **Web browser**
Allows the user to assume the role to perform actions on the resources.
- ☐ **Web console**
Allows the user to assume the role to perform actions on the resources.
- ☐ **Web console**
Allows the user to assume the role to perform actions on the resources.
- ☐ **Web console**
Allows the user to assume the role to perform actions on the resources.

Use case

When an IAM role is assumed, the user can perform actions on the resources.

While creating an IAM role, we first give who can assume this role and perform actions.

- 14
- 15
- 16
- 17
- 18
- 19
- 20

```
Permissions Trust relationships Tags Last Accessed Remove sessions

Trusted entities
Entities that can assume this role under specified conditions.
[Edit trust policy]

1: {
2:   "principal": "ec2.amazonaws.com",
3:   "condition": {
4:     "type": "StringEquals",
5:     "principal": "ec2.amazonaws.com",
6:     "action": "iam:AssumeRole",
7:     "resource": "arn:aws:iam::123456789012:role/ExampleRole"
8:   }
9: }
10: }
11: }
12: }
13: }
```

Ec2 is the trusted entity here, ec2 can assume the role and role has s3-full-access-permission.

Step 1: Select trusted entity

Step 2: Add permissions

Permission policy summary

23) What is USER Data in EC2?

Answer:

21



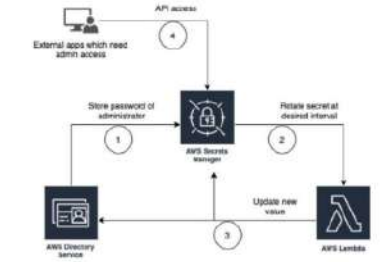
22

```
#!/bin/bash
# Use this for your user data (script from top to bottom)
# Install httpd (Linux 2 version)
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "Hi! Welcome World from http://localhost:80" > /usr/share/httpd/html/index.html
```

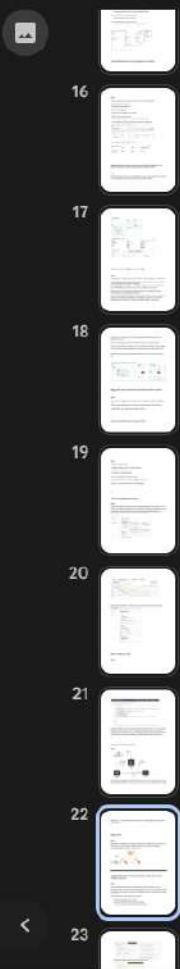
User Data in EC2 is a way to automate tasks during instance boot-up. You can provide a script (like a shell script) that installs packages, configures services, or sets up the environment. This script runs once during the first boot of the instance, making it great for initial setup or provisioning. Example, docker installation done with user data.

24) How to handle the secret in AWS?

Answer:



Use AWS Secrets Manager or AWS SSM Parameter Store to store secrets securely. Avoid hardcoding credentials in EC2 or code. You can access secrets through IAM roles using

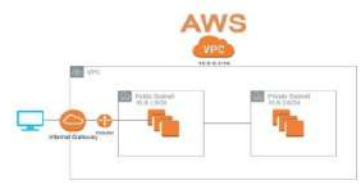


AWS SDK or CLI. Secrets Manager also allows rotation, audit logging, and fine-grained access control.

25) What is VPC?

Answer:

VPC (Virtual Private Cloud) is a logically isolated section of AWS where you can launch resources in a custom-defined IP range. It provides full control over networking, subnets, gateways, route tables, and security.



26) Suppose there are two S3 buckets and they need to share the data to each other. What will you do?

Answer:

You can configure Bucket Policy on both buckets to allow access from the other bucket's account or service principal. Use IAM roles or Cross-Account access if buckets are in different accounts. You can also use S3 Replication to sync data between buckets automatically.

If these two buckets are in different aws accounts:

- Enable versioning on both buckets.
- Create an IAM Role in Account A for Replication : This role allows S3 to assume it and replicate data.

Page 23 / 158

17

18

19

20

21

22

23

24

Test Relationship

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": "s3:putobject"
    }
  ]
}
```

Bucket Policy for Account B

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": "s3:putobject"
    }
  ]
}
```

Add Bucket Policy to Destination Bucket (Account B)

Configure Replication Rule In Account A (Source Bucket)

S3 Replication Rule

27) What are the differences between Load Balancer and why do we need them?

Answer:

AWS offers 3 types:

Load balancer types

Application Load Balancer (ALB) Network Load Balancer (NLB) Gateway Load Balancer (GLB)

ALB: This is an Application Load Balancer when you create a load balancer, you specify the protocols and ports for the load balancer to listen on. ALB supports HTTP, HTTPS, and TCP. It is a Layer 7 load balancer.

NLB: This is a Network Load Balancer when you create a load balancer, you specify the protocols and ports for the load balancer to listen on. NLB supports TCP, UDP, and TLS. It is a Layer 4 load balancer.

GLB: This is a Gateway Load Balancer when you create a load balancer, you specify the protocols and ports for the load balancer to listen on. GLB supports IPsec-encapsulated traffic. It is a Layer 3 load balancer.

Feature / LB Type	ALB (Application LB)	NLB (Network LB)	GLB (Gateway LB)
OSI Layer	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP)	Layer 3 (IP)
Protocols Supported	HTTP, HTTPS	TCP, UDP	IP (GRE/NAT-encapsulated)
Routing Type	Path-based, host-based routing	Forwarding to targets via TCP/UDP	Routes traffic to third-party virtual appliances
Use Case	Web apps, APIs, microservices	High-performance apps, static IPs	Security appliances (firewalls, IDS/IPS)
Performance	Good for application layer decisions	High performance, low latency	Scalable traffic inspection
Target Types	EC2, IP, Lambda	EC2, IP	AWS Endpoint (used by security appliances)
Static IP Support	No	Yes	Yes

* Application Load Balancer (ALB) – for HTTP/HTTPS traffic with path-based routing.

* Network Load Balancer (NLB) – for TCP/UDP traffic, high performance, low latency.

* Gateway Load Balancer (GLB) – Layer 3, used mainly in security and networking scenarios.

Left sidebar thumbnails (pages 19-26):

- 19: Diagram of traffic distribution
- 20: AWS IAM console screenshot
- 21: AWS IAM console screenshot
- 22: AWS IAM console screenshot
- 23: AWS IAM console screenshot
- 24: AWS IAM console screenshot
- 25: AWS IAM console screenshot (highlighted)
- 26: AWS IAM console screenshot

Main content area (Page 25):

They distribute traffic, ensure high availability, and support auto scaling.

28) What is the meaning of assume Role in IAM?

Answer:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

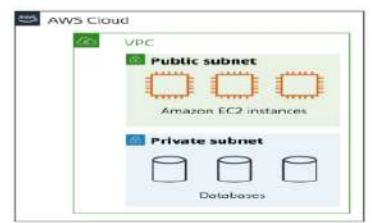
"Assume Role" allows a user or service to temporarily take on a role with specific permissions. It's useful for cross-account access, EC2 access to other AWS services, or delegated permissions. This is enabled through trust relationships and "sts:AssumeRole" API.

29) What is the uses of Subnet?

Answer:

Subnets divide a VPC into smaller IP address blocks. They allow logical separation of resources, such as public-facing and internal resources. Public subnets are connected to the Internet Gateway; private ones are not. Subnets also help with security control, routing, and AZ-level resource placement.

- 20
- 21
- 22
- 23
- 24
- 25
- 26
- 27



30) What is the way to preinstall the required software on your server before it comes up in running state?

Answer:

Use EC2 User Data to run shell scripts at instance launch. You can install packages, configure apps, or pull code from GitHub. It ensures the server is ready to use as soon as it boots up, automating manual setup.

```
Allow tags in metadata info
Select

User data info
# /bin/bash
# Use this for your user data (script from top to bottom)
# install httpd (Linux 2 version)
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Hello World from $hostname" > /var/www/html/index.html
```

31) How can I reassign the elastic IP address to another server ?

OR, Your production EC2 instance (app-server-1) is attached to an Elastic IP (EIP-1). Suddenly, the server faces a critical issue (e.g., disk failure or OS-level corruption), and you need to quickly switch over to a backup server (app-server-2) to restore services. The application's DNS (app.mycompany.com) is pointing to the Elastic IP. How will you reassign the Elastic IP to the new instance with minimal downtime?

Explain the uses of EIP in EC2?

Answer:

```
graph LR; Domain --> EIP; EIP --> EC2A[EC2 A]; EIP --> EC2B[EC2 B];
```

You can keep and reassign this IP to any EC2.

- 1 Click "Actions" → **Disassociate Elastic IP address**.
- 2 Associate to another ec2 you want.

EIP (Elastic IP) is a static, public IPv4 address provided by AWS. It doesn't change when the instance stops or restarts. It's useful for stable DNS entries and for ensuring users can always reach your instance. You must explicitly associate it to your EC2, and remember—EIP is chargeable if not attached to a running instance.

Name	Allocated Public IPv4 address	Type	Allocation ID	Source/DNS entry
My EIP	15.204.133	Public IP	igw-081980707030811	

32) What are the different types of EC2?

Answer:

EC2 types are grouped into families:

* T Family (e.g. "t3.micro") → General purpose

* C Family ("c5.large") → Compute optimized

* R Family ("r5.large") → Memory optimized

Others include M (balanced), P (GPU), I (storage), etc. Choose based on your workload needs.

33) Have you experienced any challenging issue in your project? How did you or your team resolve it?

Answer:

(Example) We once faced an issue where our EC2-based application was intermittently failing due to memory leaks. We used CloudWatch metrics to detect the spike patterns, added swap space temporarily, and later containerized the app to better isolate and restart processes. Load testing helped us validate the fix.

34) Application on EC2 not showing images from S3? Troubleshooting steps?

Answer:

Check:



Search



ENG

IN



12:13

17-05-2026

23

24

25

26

27

28

29

30

1. EC2 IAM role has "s3:GetObject" permissions

2. S3 bucket policy allows read access from EC2

3. Network connectivity or VPC endpoint issues

4. Check if image URLs are correct and public (if required)

5. Ensure the S3 bucket objects are not deleted or expired

35) Name all AWS services will you consider while setting up CI/CD pipeline?

Answer:

- * CodeCommit or GitHub for source control
- * CodeBuild for build & test
- * CodeDeploy for deployment
- * CodePipeline for orchestration
- * S3 for artifact storage
- * CloudWatch for monitoring
- * IAM for access control

36) How can you restore deleted S3 data?

Answer:

Enable versioning and MFA delete on your bucket. If enabled, you can restore deleted objects by removing the delete marker or restoring a previous version. Without versioning, deleted data is permanently lost and cannot be recovered.

- 24
- 25
- 26
- 27
- 28
- 29
- 30
- 31

marishdmed11

Objects 1/1

Name	Type	Version	Last modified	Size	Storage class
bucket-27912211-us-east-1	Bucket	bucket-27912211-us-east-1	2023-05-15 10:00	0 B	Standard

37) What is a Target Group?

Answer:

A Target Group is part of a load balancer setup. It defines the destination (EC2, IP, Lambda) to which the load balancer routes traffic. You can configure health checks, port numbers, and attach multiple instances to it.



38) You have an Auto Scaling Group with a default desired capacity of 2 instances. You create a scheduled action to increase the desired capacity to 5 at 9 AM every weekday, and another one to scale it down to 2 at 6 PM.

However, during one weekday, a target tracking policy (based on CPU > 70%) increased the desired capacity to 6 at 10 AM.

25

26

27

28

29

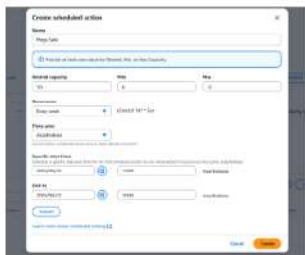
30

31

32

What happens at 6 PM that day? Will the ASG scale back to 2, stay at 6, or do something else? Why?

Answer:



At 6 PM, the scheduled action will override the current desired capacity and set it back to 2, as defined.

Even though the target tracking policy increased it to 6 earlier based on CPU, scheduled actions take priority at their scheduled time and reset the desired, min, and max values as configured.

This highlights that scheduled actions do not consider real-time metrics, they apply the specified capacity regardless of actual load.

Note: Target Tracking resumes monitoring after that and will scale back up if the high CPU persists. There may be a temporary performance degradation between the scale-down and scale-back-up.

39) What is the use of CloudWatch Alarms and CloudWatch Events?

Answer:

- * Alarms: Trigger actions (like auto scaling or notifications) based on CloudWatch metrics (CPU, memory, etc.)
- * Events: Respond to AWS service changes (e.g., EC2 start/stop, CodePipeline changes) and trigger Lambda functions or Step Functions. Together, they automate and monitor resource behaviour.

40) How no of subnets are calculated based on CIDR block?

1. Understand what CIDR /16 means: starting 16 bits fixed.
 - VPC CIDR: 10.0.0.0/16
 - IPs in VPC = $2^8 \times (32 - 16) = 65,536$ total IPs
2. Subnet size = /24: first 24 bits fixed remaining 8 can be used for ips.
 - Each subnet = $2^8 = 2^8 (32 - 24) = 256$ IPs
 - But AWS reserves 5 IPs, so 251 usable IPs per subnet
3. Bits borrowed = $24 - 16 = 8$ bits
 - So, you can create $2^8 = 256$ subnets inside the /16 VPC ranging in series like 10.0.0.x/24 ... 10.0.1.x/24 .. 10.0.2.x/24 10.0.255.x/24

Final Answer:

From a /16 VPC (10.0.0.0/16), you can create:

1. 256 subnets of size /24
2. Each subnet has 256 total IPs, and 251 usable

41) What is DNS?

DNS = Domain Name System.

It translates domain names (like amazon.com) into IP addresses (like 192.0.2.44).

Your browser uses DNS to find the correct server to connect to.

AWS offers Route53 as a managed DNS service.

42) How to increase EBS storage automatically?

1. Enable EBS Auto-Scaling using CloudWatch + Lambda.

2. Steps:

- Create a CloudWatch alarm for low disk space (eg, $\leq 10\%$).
- Trigger a Lambda function to:
 - a. Modify the volume size using `modify-volume`.
 - b. Run `grouppart` and `resizefs` on EC2 (via SSM or script).

3. You can also use EC2 Auto Recovery to resize on boot (less common).

Note: The file system must be resized **after** increasing EBS.

For xfs filesystem,

```
sudo grouppart /dev/nvme1n1 1
sudo xfs_growfs /data
```

For ext4 filesystem

```
sudo grouppart /dev/nvme1n1 1
sudo resize2fs /dev/nvme1n1p1
```

Filesystem type can be checked with command `df -Th`

43) What is DHCP?

1. DHCP – Dynamic Host Configuration Protocol.
2. It automatically assigns IP addresses to devices on a network.
3. In AWS, your VPC has a built-in DHCP server.
4. DHCP options set: Domain name, DNS servers, NTP servers
5. EC2 gets its private IP via DHCP during boot.

44) How does ssh work while connecting ec2 instance?

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

28

29

30

31

32

33

34

35

1. You use the **SSH private key (.pem)** stored locally.

2. EC2 has the **public key** stored in `~/ssh/authorized_keys`.

3. When you run:

```
ssh -i key.pem ec2-user@EC2-IP
```

- The client proves it has the private key.
- The EC2 verifies it matches the stored public key.
- If valid, connection is allowed.

45) How do you ensure security of our EKS cluster? - MFA, secrets, Role-Based Access Control (RBAC), use rolling update strategy, PodSecurityPolicy

1. **IAM + RBAC**: Map IAM roles to K8s users with fine-grained permissions.
2. Use **Secrets Manager** or SSM for storing credentials.
3. **MFA** (Multi-Factor Auth) for all AWS IAM users.
4. Restrict public access to EKS endpoint (use private access only).
5. Network Policies: Control pod-to-pod communication.
6. Pod Security: Use PodSecurityPolicy or OPA Gatekeeper.
7. Encrypted etcd and use **KMS** for secret encryption.
8. Use **rolling updates** to avoid zero-day issues.
9. Monitor with CloudTrail, GuardDuty, and Audit Logs.

46) cooldown period in Autoscaling group

1. A cooldown is a **waiting time after a scaling activity**.
2. It ensures ASG doesn't trigger multiple scaling actions too quickly.
3. Default cooldown is 300 seconds.
4. Example: After adding a new instance, cooldown prevents adding another too soon, giving time for CPU/memory to stabilize.
5. Two types:
6. Default cooldown (for ASG)
7. Scaling policy cooldown (overrides default)

47) what is your day-to-day task as aws administrator?

1. Monitor EC2, RDS, S3, Load Balancer health via CloudWatch.
2. Handle IAM role and policy management for users and services.
3. Take backups (snapshots) of EBS and RDS.



Search



ENG

IN

12:13

17-05-2026

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Page 35 / 158

75%

29

30

31

32

33

34

35

36

4. Manage Auto Scaling Groups, AMIs, and launch templates.

5. Resize EBS volumes or EC2 types based on utilization.

6. Implement S3 lifecycle policies and encryption.

7. Rotate secrets and manage SSM Parameters.

8. Review cost reports and optimize usage.

9. Apply patches using Systems Manager (SSM).

10. Troubleshoot connectivity, DNS, and SSH login issues.

48) What are SCPs (Service Control Policies) in AWS Organizations?

Answer:

1. SCPs define **max permission boundaries for AWS accounts**.

2. They don't grant permissions but limit what IAM roles/users can do.

3. They apply to all users, even root, within the organization or OU.

4. Useful for compliance and governance — e.g., block S3 DeleteBucket.

■ Example use case: Block all EC2 creation in dev accounts.

49) What is the difference between ECS, EKS, and Fargate?

Answer:

1. **ECS** = AWS-native container service.

2. **EKS** = Managed Kubernetes by AWS.

3. **Fargate** = Serverless runtime for containers (used with ECS/EKS).

4. ECS is simple, EKS is for Kubernetes users, Fargate is for not managing EC2.

■ Real-world:
ECS = AWS truck
EKS = Bring your own truck (Kubernetes)
Fargate = Rent a driverless car (no server to manage)

50) What is the use of Read Replica in RDS?

Answer:

1. A read replica is used to offload read traffic from the main (primary) RDS database.

2. It is not a backup — backups and snapshots are used to restore data.

A read replica is a live, continuously updated copy of the main database.

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

30

31

32

33

34

35

36

37

3. It uses asynchronous replication, so there might be a small delay between master and replica.

4. Purpose:

- * Handle read-heavy applications (e.g., dashboards, analytics)
- * Improve performance and reduce load on the primary DB
- * Support scaling reads across multiple regions (cross-region replica)
- * Optionally promote to standalone DB in case of disaster (manual failover)

5. and, the read replica has the same username and password as the main DB — it's part of the replication.

6. The endpoint includes replica in its name so you can target read queries to it in your app separately.

7. You can't write to the read replica — it's read-only.

...

###Notes/Pointer you should focus while talking about this question:

- * Backup = point-in-time restore
- * Snapshot = manual backup image
- * Read replica = real-time read-only live copy to reduce primary load and scale reads

KUBERNETES Interview Questions & Answers

1) Explain the architecture of Kubernetes.

31

32

33

34

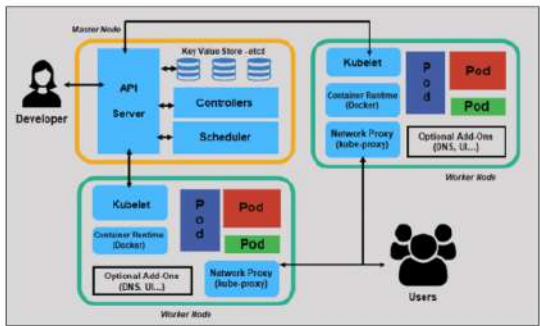
35

36

37

38

Answer:



The diagram illustrates the Kubernetes Master-Worker architecture. On the left, a 'Developer' icon points to the 'Master Node'. The 'Master Node' (Control Plane) contains four main components: 'API Server', 'etcd' (Key Value Store), 'Controllers', and 'Scheduler'. These are interconnected. The 'API Server' is the central hub. The 'etcd' store holds cluster data. 'Controllers' manage the state of the cluster, and the 'Scheduler' assigns pods to nodes. On the right, 'Worker Nodes' are shown. Each 'Worker Node' contains 'Kubelet' (Container Runtime/Decker), 'Pod' (represented by a blue box with 'Pod' text), 'Network Proxy (kube-proxy)', and 'Optional Add-Ons (DNS, UI, ...)'. The 'Kubelet' on the worker node communicates with the 'API Server' on the master node. The 'Network Proxy' on the worker node handles network traffic. 'Users' are shown interacting with the 'Network Proxy' on the worker node. The diagram also shows 'Pod' icons on the worker nodes, indicating that pods are scheduled to run on these nodes.

Kubernetes follows a master-worker architecture. The control plane consists of components like the API server (cluster entry point), etcd (cluster state store), controller manager, and scheduler. The worker nodes run the Kubelet (talks to API server), kube-proxy (handles networking), and actual containers inside pods. Users interact using "kubectl" or YAML manifests. The scheduler assigns pods to nodes, and the controller ensures the cluster's actual state matches the desired state (continuously reconcile the actual state with the desired state declared in manifests (kubectl apply)).

Master Node (Control Plane) Components

1. **API Server** – Exposes the Kubernetes API and acts as the communication hub.
2. **etcd** – Distributed key-value store holding all cluster data/state.
3. **Controller Manager** – Runs controllers to maintain desired cluster state.
4. **Scheduler** – Assigns pods to suitable nodes based on resources/policies.

Worker Node Components

1. **Kubelet** – Ensures containers are running in pods as instructed by the control plane.
2. **Kube-Proxy** – Manages networking rules and enables pod-to-pod/service communication.



Q2) Your application is deployed on AWS EKS cluster and after deployment you see that your pod is crashing again and again. What can be the possible causes or issues that we see with pod crashing ? And how do you troubleshoot it?
OR

CashLoopBackOff

- Application can crash itself (e.g., uncaught exception)
- Required environment variables are missing
- Misspecified command or arguments
- Missing configuration files or secrets
- Bad/extra permissions (such as volume)

Index	Offset	Value
Index 12	Describe post-quiz name	
Index 13	Quiz name	

Meaning

- Image name or tag is incorrect (e.g., typos)
- Image does not exist in the registry
- Private image without a configured `ImagePullSecret`
- Network/DNS misconfiguring access to registry
- Cluster rate limits on public images

bioRxiv preprint doi: <https://doi.org/10.1101/000000>; this version posted January 1, 2016. The copyright holder for this preprint (which was not certified by peer review) is the author/funder, who has granted bioRxiv a license to display the preprint in perpetuity. It is made available under aCC-BY-NC-ND 4.0 International license.

- Verify the image name and tag
- Read image pull secret for private registries
- Ensure the node can reach the registry endpoint(s) [CNI]

Type	Reason	Age	File	Message
Normal	Started	0m	default-scheduler	Successfully assigned default/flash-employee-4-78605056-181 to node00
Normal	Succeeded	6m 10s (over 2m)		Normal container flash-employee
Normal	Killed	2m 10s (over 2m)	node00	Container flash-employee failed: OOMKilled, will be restarted
Normal	Killed	2m 10s (over 2m)	node00	Container flash-employee failed: OOMKilled, already present in backlog
Normal	Succeeded	6m 10s (over 2m)	node00	Container container flash-employee
Normal	Succeeded	6m 10s (over 2m)	node00	Employee node00: still active. Flash with successfully: 0
Normal	Succeeded	6m 10s (over 2m)	node00	Back up container flash-employee: flash-employee in pod flash-employee-4-78605056-181

If the liveness probe fails, Kubernetes kills the container and restarts it (based on the Pod's restart policy).

Q3 LIVENESS PROBE AND READINESS PROBE

1. Liveness Probe

Purpose:

Checks whether the application inside the container is still alive (healthy).

If the liveness probe fails, Kubernetes kills the container and restarts it (based on the f

Think of it as:

"Is my app still working, or is it stuck/frozen?"

Example:

```
yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Scenario:

- Suppose your Flask app hits a deadlock or memory issue.
- The app process is still running but not responding.
- Liveness probe fails → kubelet restarts the container → app comes back healthy.



Search



ENG
IN

12:13
17-05-2026

2. Readiness Probe

Purpose:

Checks whether the application is ready to serve traffic.

If the readiness probe fails, Kubernetes stops sending traffic to that Pod (removes it from the Service's Endpoints list), but doesn't restart the container.

Think of it as:

"Can my app handle requests right now?"

Example:

```
yaml
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
```

Scenario:

- When your app starts, it may take time to connect to a database.
- Until it's ready, readiness probe fails → Service won't route requests to it.
- Once `/ready` returns success → Pod is added back to load balancer.

Q4) If your application gives a 403 error, what could be the causes?

Answer:

- A 403 Forbidden error means the client is authenticated but is not authorized to access the requested resource.
- It's a server-side rejection of the request due to lack of permissions.

A 403 (Forbidden) can occur due to multiple reasons:



Search



ENG

IN

12:13

17-05-2026

37

38

39

40

41

42

43

44

Answer:

```
[ then / browser ]
{
  https://customer.yourdomain.com
}

Node.js -> npm install (in or -fpm) printing on file

AS loadBalancer (ILB) -> created by service of type loadBalancer

customer-service (type LB)
selector: app=customer-prod

customer-api Deployment Pods
Label: app=customer-prod
```

Dockerfile

```
Dockerfile
FROM node:18
WORKDIR /app
COPY server.js .
RUN npm install express
CMD ["node", "server.js"]
```

Build and push to Docker Hub/ECR:

```
bash
docker build -t riski/customer-api:latest .
docker push riski/customer-api:latest
```

38

39

40

41

42

43

44

45

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: customer-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: customer-api
  template:
    metadata:
      labels:
        app: customer-api
    spec:
      containers:
        - name: customer-container
          image: public.ecr.aws/customer-api:latest
          ports:
            - containerPort: 8080
```

```
apiVersion: v1
kind: Service
metadata:
  name: customer-service
spec:
  type: LoadBalancer
  selector:
    app: customer-api
  ports:
    - port: 80
      targetPort: 8080
      protocol: TCP
```

Apply on EKS

```
kubectl apply -f customer-deployment.yaml
kubectl apply -f customer-service.yaml
```

Wait a few seconds and run:

```
kubectl get svc customer-service
```

You will see something like:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	IP
customer-service	LoadBalancer	10.0.1.10	40003240071438-us-east-1.elb.amazonaws.com	80/TCP	

You can expose an application using a Service of type LoadBalancer, which automatically provisions an AWS ELB. Expose it to the internet via an AWS Load Balancer automatically created by the Service. Use Route53 for DNS management and ACM for TLS certificates. Also, configure security groups and IAM roles appropriately.

- 1 Create Dockerfile, build and push image to ECR or Docker hub.
- 2 Write k8 Deployment and Service [of type load balancer] files.
- 3 Apply k8 files. This will create pods of application and a lb on AWS.
- 4 From route 53, map domain to lb domain.
- 5 Access your application via http://you-domain.

Q8) How to expose k8 application with microservices to outer world with nginx ingress controller? Explain with service files and examples.

Answer:

```
graph TD
    Domain[https://domain.yourdomain.com] --> Ingress[Ingress]
    Ingress --> ALB[ALB LoadBalancer]
    ALB --> IngressController[Ingress Controller]
    IngressController --> LoginService[login-service]
    IngressController --> ProductService[product-service]
    LoginService --> LoginPods[login-pods]
    ProductService --> ProductPods[product-pods]
```

1. Create deployment and service of type ClusterIP for each microservice[login and product].
2. This will create your pods product_pods and a service which route request to these pods. Login_pods are also created and a service login_service which provides static ip to these pods internally.
3. Create an ingress file to route to both these services. Rule-book. Apps should handle being served from sub-paths (/login, /product) — if not, remove rewrite-target annotation or set it to the correct path
3. Install Nginx ingress controller. This will create:
 - ingress-nginx namespace
 - ingress-nginx-controller Deployment [ingress-controller pods]
 - LoadBalancer Service (which creates an ELB on AWS)
 - RBAC, ConfigMaps, etc.
4. Since the ingress-nginx-service is of type load-balancer, it creates a lb on AWS. K8 assigns a additional NodePort to this loadbalancer service. Now, this ingress-nginx-service says whatever traffic comes at NodePort , forwards it to ingress-controller pods. K8 adds all the worker nodes in the cluster as target to this lb. And whatever traffic lb receives, it routes it to any random node on NodePort. So traffic is coming at nodeport

on any node in the cluster which is routed to ingress controller pods.
These ingress controller pods read ingress file rules and route it to underneath clusterip service which routes to the specific pod. So now, from http://load_balancer_domain/product will reach to product pods and http://load_balancer_domain/login will reach to login pods.

5. In route 53, add the domain of the load balancer on AWS.

```
# login-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: login-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: login
  template:
    metadata:
      labels:
        app: login
    spec:
      containers:
        - name: login
          image: nginx
          ports:
            - containerPort: 80

# products-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: products-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: products
  template:
    metadata:
      labels:
        app: products
    spec:
      containers:
        - name: products
          image: nginx
          ports:
            - containerPort: 80

# login-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: login-service
spec:
  selector:
    app: login
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP

# products-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: products-service
spec:
  selector:
    app: products
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

kubectl apply -f login-deployment.yaml

kubectl apply -f products-deployment.yaml

Step 2: Install the NGINX Ingress Controller (Static Manifest)

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/master/deploy/static/provider/aws/ingress-nginx.yaml
```

This creates:

- ingress-nginx namespace
- ingress-nginx-controller Deployment
- loadbalancer-service (which creates an AWS ELB)
- RBAC, ConfigMap, etc.

kubectl apply -f <https://raw.githubusercontent.com/kubernetes/ingress-nginx/master/deploy/static/provider/aws/ingress-nginx.yaml>

41

42

43

44

45

46

47

48

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: / # optional if path is handled in app
spec:
  rules:
  - host: example.com
    http:
      paths:
      - path: /login
        pathType: Prefix
        backend:
          service:
            name: login-service
            port:
              number: 80
      - path: /products
        pathType: Prefix
        backend:
          service:
            name: products-service
            port:
              number: 80
```

Flow Explanation:

1. Route 53 maps `demo.yourdomain.com` to the AWS Load Balancer DNS name created by the `Ingress Controller`.
2. The `AWS LoadBalancer (ALB/NLB)` forwards the request to the EKS cluster entry.
3. `Ingress Controller` inside the cluster listens to the rules defined in `Ingress` objects.
4. Based on host/path (e.g., `/login`, `/products`), it forwards the request to the right `Kubernetes Service`.
5. The service sends the request to the appropriate backend `Pod`.

...

Q7) Do ingress and service have same work? What is the difference between Ingress and Service?

42

43

44

45

46

47

48

49

Answer:

2. Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /login
        pathType: Prefix
        backend:
          service:
            name: login-service
            port:
              number: 80
      - path: /cart
        pathType: Prefix
        backend:
          service:
            name: cart-service
            port:
              number: 80
```

Option 1 Use Service of type LoadBalancer

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

Both can work together: Ingress routes traffic to Services, which connect to Pods.

Service

- A Service is used inside the Kubernetes cluster.
- It groups a set of pods and provides a stable IP and port to access them.
- Think of it like a door inside the cluster to reach the right set of pods.
- Example: login-service routes to 3 login pods.

Ingress

- An Ingress is like a traffic manager at the main gate of the cluster.
- It reads rules like:
 - "If the request is for /login on demo.yourdomain.com, send it to login-service."
- It requires an Ingress Controller to work (like Nginx Ingress Controller).

Q8) What are the different types of services in k8?

Answer:

A Service in Kubernetes provides a stable network endpoint for a dynamic set of pods. A Service exposes a set of Pods to other services or external users.

Four Types of services :

- * ClusterIP (default) for internal access.
- * NodePort for exposing via static port on each node
- * LoadBalancer for external cloud access. This ensures reliable communication even when pods die or get recreated.
- * ExternalName

Q9) How request flows & to which component when we apply a deployment in K8 ?

Answer: In Kubernetes,

1. You run: `kubectl apply -f deployment.yaml (replicas=3)`
 - |
 - v
2. API Server stores Deployment in etcd
 - |
 - v
3. API Server sends "Deployment ADDED" event to Deployment Controller
 - |
 - v
4. Deployment Controller creates a ReplicaSet (POST ReplicaSet)
 - |
 - v
5. API Server stores ReplicaSet in etcd and sends "RS ADDED" event

44

45

46

47

48

49

50

51

6. ReplicaSet Controller sees RS wants 3 replicas → creates 3 Pod objects

- POST Pod #1
- POST Pod #2
- POST Pod #3

7. API Server stores 3 Pod objects (Pending) in etcd

8. Scheduler receives "Pod ADDED (no nodeName)" events

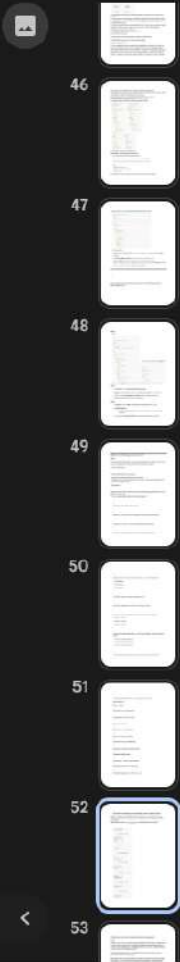
9. Scheduler assigns nodeName to each Pod (PATCH pod.nodeName)

- Pod #1 → node-2
- Pod #2 → node-5
- Pod #3 → node-1

10. API Server updates Pods in etcd → sends "Pod MODIFIED" event to respective kubelets

- node-2 kubelet sees Pod #1
- node-5 kubelet sees Pod #2
- node-1 kubelet sees Pod #3

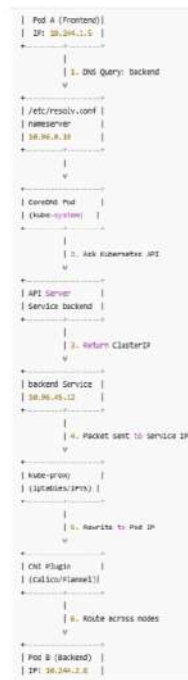
11. Each Kubelet pulls image & starts containers → Pod becomes Running



Q10) Difference between coredns, kubeproxy and cni component in K8 ?

Answer: In Kubernetes, CoreDNS resolves service names to ClusterIP, kube-proxy translates ClusterIP to Pod IP, and CNI ensures packets can actually reach that Pod IP across nodes.

Resolve service domain "http://backend" to IP address = work of CoreDNS



Q11) How does a pod communicate across different namespaces?

Answer:

By default, pods can communicate across namespaces unless network policies restrict it. You can reach a pod in another namespace using its fully qualified domain name: `"service-name.namespace.svc.cluster.local"`. For finer control, use NetworkPolicies to allow or deny cross-namespace traffic.

Like frontend pod wants to talk to a backend pod with service name as backend = `backend.production.svc.cluster.local`

The syntax `service-name.namespace.svc.cluster.local` is not written in a specific file by default — It's a convention used by Kubernetes DNS to resolve services inside the cluster. any pod in any namespace can resolve the service via: `service-name.namespace.svc.cluster.local`

Q12) How do you manage storage in case of kubernetes? OR Where does PV exist in Kubernetes?

Answer:

```
graph TD
    Pod[Pod] --> User[User]
    User --> PVC[PVC]
    PVC -- "requests storage (size, access)" --> Bound[Bound to]
    Bound --> PV[PV]
    PV -- "actual storage with path (e.g., NFS path or S3)" --> Backed[backed to]
    Backed --> Storage[NFS or S3]
```

Example:

You make a phone call to your mom. Phone call = PVC you request for storage
Saved Contact - pointing to phone number - PV pointing to actual storage
Actual number can be saved to phone itself or to Gmail account
Actual storage can be on same node or outside cluster.

PV is not the storage itself but rather an abstraction (pointer) in k8 that represents piece of storage which is provisioned for use by pods.

A PersistentVolume (PV) is a cluster-managed storage resource, either static or dynamically provisioned.

A PersistentVolumeClaim (PVC) is a request from a pod to use some amount of storage. Once a PVC matches a PV, the pod mounts the volume. This setup decouples storage from pods, ensuring persistent data across pod restarts. PVs can be backed by AWS EBS, NFS, or other CSI drivers.

PV:

- It is **created by the admin or dynamically by Kubernetes**.
- Think of it like a USB drive — available for use.

PVC is a **request** from a pod/application for storage.

Pod → PVC → PV → NFS/Disk

Pod uses PVC, PVC binds to PV, and PV is just the path to actual disk (like EBS, NFS, etc.)

Q13) What do you understand by Internal and External storage while working with nodes?

Answer:

Internal storage is local to the node (like instance store) and is ephemeral — it gets lost if the node dies.

External storage is provisioned externally (like AWS EBS, EFS) and persists beyond pod or node lifecycle. External storage is necessary for stateful workloads like databases, while internal storage is fine for temporary or cache data.



Search



ENG
IN

12:16
17-05-2026

Q14) How do you handle Kubernetes cluster control plane upgrade and worker node upgrade? [IMP CONCEPT]

Answer:

- 1. Control Plane Upgrade First**
We start by upgrading the control plane (i.e., the master components like API server, controller-manager, scheduler, etc.).
This is because the control plane manages the cluster and needs to be compatible with the newer version before upgrading the nodes.
- 2. Creating a New Node Group for Worker Nodes**
Instead of upgrading the existing worker nodes directly, we followed a blue-green deployment approach:
 - We created a new node group with the upgraded version (like a new EC2 Auto Scaling Group in case of AWS).
 - This node group was launched with the new AMI version that included the updated Kubernetes version and all required dependencies.
- 3. Draining and Migrating Workloads**
Once the new node group was ready:
 - We gradually drained the old nodes (using `kubectl drain`) to safely evict the pods.
 - The pods were rescheduled automatically onto the new upgraded nodes.
- 4. Deleting the Old Node Group**
After verifying that all workloads were running smoothly on the new node group:
 - We deleted the old node group, effectively completing the upgrade.

3. Taint + Cordon + Drain Strategy
Instead of just tainting the node:

- First run `kubectl cordon <node-name>` → this prevents new pods from being scheduled.
- Then `kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data`
 - This evicts the pods safely to other nodes before tainting/upgrading.
- Finally, perform upgrade, remove taint, and uncordon after the upgrade.

4. Use Node Affinity + Tolerations Carefully

- Ensure pods have tolerations only when needed.
- Use anti-affinity to spread pods across nodes.



50



51



52



53



54



55



56



57



1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 2680, 26

1000

There is a data processing application which require heavy nodes [Memory-optimized → r5] How will you make sure that inside the same cluster, it is managed? Normal application with t3.medium size nodes and this Data Processing app co-exist in your cluster? OR

Answer:

TEST

```

% A table of authors: type=author:web:chubila
code/codem: Talited

```

```

void
myWidget::init()
{
    QHBoxLayout *hLayout = new
        QHBoxLayout;
    QVBoxLayout *vLayout =
        new QVBoxLayout;
    QWidget *w1 = new
        QWidget;
    w1->setObjectName("w1");
    w1->setGeometry(
        10, 10, 100, 100);
    w2 = new
        QWidget;
    w2->setObjectName("w2");
    w2->setGeometry(
        110, 10, 100, 100);
    w3 = new
        QWidget;
    w3->setObjectName("w3");
    w3->setGeometry(
        220, 10, 100, 100);
    w4 = new
        QWidget;
    w4->setObjectName("w4");
    w4->setGeometry(
        330, 10, 100, 100);
    w5 = new
        QWidget;
    w5->setObjectName("w5");
    w5->setGeometry(
        440, 10, 100, 100);
    w6 = new
        QWidget;
    w6->setObjectName("w6");
    w6->setGeometry(
        550, 10, 100, 100);
    w7 = new
        QWidget;
    w7->setObjectName("w7");
    w7->setGeometry(
        660, 10, 100, 100);
    w8 = new
        QWidget;
    w8->setObjectName("w8");
    w8->setGeometry(
        770, 10, 100, 100);
    w9 = new
        QWidget;
    w9->setObjectName("w9");
    w9->setGeometry(
        880, 10, 100, 100);
    w10 = new
        QWidget;
    w10->setObjectName("w10");
    w10->setGeometry(
        990, 10, 100, 100);
    hLayout->addWidget(w1);
    hLayout->addWidget(w2);
    hLayout->addWidget(w3);
    hLayout->addWidget(w4);
    hLayout->addWidget(w5);
    hLayout->addWidget(w6);
    hLayout->addWidget(w7);
    hLayout->addWidget(w8);
    hLayout->addWidget(w9);
    hLayout->addWidget(w10);
    vLayout->addLayout(hLayout);
    setLayout(vLayout);
}

```

Taints are applied to nodes to prevent pods from scheduling unless they tolerate the taint. Tolerations are applied to pods so they can run on tainted nodes. This is useful for isolating special workloads (e.g., GPU workloads, system-critical apps). It's a way to guide Kubernetes' scheduling decisions with more control.

Example,
Taint = 4 Crore label attached to a flat. Taint attached to flat. So any random person cannot stay in it. [No random pod can be scheduled on it]
Toleration = Only person [pod] having 4 crore [matching toleration] can live in it.

...

Q16) Can you explain the concept of Pod Affinity and Node Affinity?

Answer:
Example of Pod Affinity:
A student wants to live in a flat which has students.
Student = pod flat = node students = other similar pods with same labels.
A pod wants to be scheduled on node which have similar pod with matching label.

Example of Node Affinity:
A businessman wants to live in "Gokuldham society".
businessman = [pod]
gokuldham society = a particular node with a label
A pod wants to be scheduled on node which has label defined in pod's yaml file.

52

53

54

55

56

57

58

59

1. Pod Affinity YAML Example

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-affinity
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels:
              app: backend
          topologyKey: "kubernetes.io/hostname"
  containers:
    - name: nginx
      image: nginx
```

Explanation:

- This pod will be scheduled on the **same node** as other pods with label **app=backend**.
- TopologyKey** defines grouping — here it's by node.

2. Node Affinity YAML Example

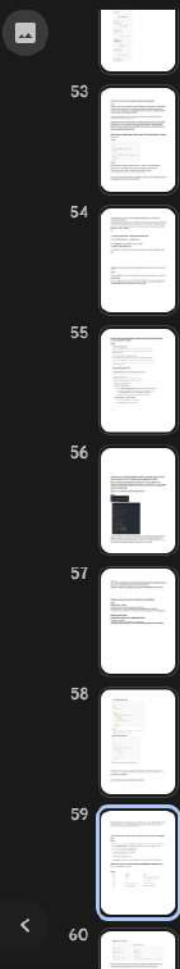
```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: node
                operator: In
                values:
                  - node1
  containers:
    - name: nginx
      image: nginx
```

Explanation:

- This pod will only be scheduled on nodes with the label **node=node1**.

Pod Affinity allows pods to be scheduled on nodes where specific other pods are running (e.g., co-locate frontend and backend). Place this pod on the same node as a pod with label **app=staging**.

Pod Anti-Affinity keeps pods apart for high availability.



Node Affinity schedules pods based on node labels (e.g., only to GPU-enabled nodes). "Place this pod on a node with label type=gpu-node". These constraints improve performance, reliability, and fault-tolerance.

...

17) Why should we use Secrets when encrypted data can be stored in ConfigMap too?

Answer:

Even though you can **encrypt data manually** in a ConfigMap, Kubernetes does **not treat it as sensitive** — it gets stored **base64-encoded**, not encrypted, and anyone with access to the ConfigMap can see it.

On the other hand, **Secrets are specially handled**:

- Stored in etcd with encryption (if enabled).
- Access is restricted by RBAC.
- Automatically masked in logs or UIs.

So, use **Secrets** for security best practices. ConfigMap is for general config, not for confidential data.

18) How do you store environment variables in k8? OR What is the different use case of ConfigMap and Secrets?

Answer:

Feature	ConfigMap	Secret
Data Type	Plaintext	Sensitive data (passwords, tokens, etc)
Encoded?	No	Yes (Base64 encoded)
Use Cases	App configs, URLs, filenames	Passwords, API keys, TLS certs
Security	No encryption by default	More secure, access controlled

54

55

56

57

58

59

60

61

ConfigMap (app-config.yaml)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  ENV: "prod"
  LOG_LEVEL: "debug"
```

Secret (app-secret.yaml)

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type:Opaque
data:
  DB_PASSWORD: MlZMc2lnbW000 # base64 of "mysecretpass"
```

Secrets are base64 encoded and optionally encrypted in etcd. While both provide configuration decoupling, Secrets offer access control and data protection.

- Use **ConfigMap** for non-sensitive configuration like ENV, URLs, flags, eg Store database hostnames
- Use **Secrets** for sensitive values like passwords, api keys, tokens, tls certificates.
- Both can be injected as **environment variables** or **mounted as files**

19) By what ways can you deploy application in Kubernetes? What kind of deployment objects you can create in k8? OR What do you understand by DaemonSet and Deployment?

Answer:

Sample Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

Sample DaemonSet YAML

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-agent
spec:
  selector:
    matchLabels:
      app: log-agent
  template:
    metadata:
      labels:
        app: log-agent
    spec:
      containers:
        - name: fluentd
          image: fluent/fluentd:latest
          resources:
            limits:
              memory: "200Mi"
              cpu: "500m"
```

Deployment:

- Ensures a specified number of pod replicas are running.
- Supports rolling updates, self-healing, rollback, and scaling.
- Used for stateless apps.

DaemonSet:

- Ensures one pod per node.
- Ideal for log collectors, monitoring agents (e.g., Fluentd or Prometheus Node Exporter).

- Connect to cluster through aws cli

• Note: `aws eks update-kubeconfig --region <region> --name <cluster-name>`

• `aws eks update-kubeconfig --region us-east-1 --name my-eks-cluster`
📄 Downloads the cluster info.

• 📄 Updates (or creates) your local kubeconfig (~/.kube/config) with this cluster context.

• Sets the **current context** to your cluster.

20) In which scenario should we use Deployment, DaemonSet, and StatefulSet?

Answer:

Comparison Table: Deployment vs DaemonSet vs StatefulSet

Feature	Deployment	Daemonset	StatefulSet
Replica management	Yes (rollback field)	No (1 pod per node automatically)	Yes (replicas field)
Pod identity	Not preserved	Not preserved	Stable identity (pod id, pod IP, ...)
Storage	Shared/ephemeral	Usually ephemeral	Persistent per pod (via PVC template)
Pod hostname	Random	Random	Predictable (statefulsetname-ordinal)
Use case	Web apps, microservices	Monitoring/logging/security agents	Databases/queues/clustered systems
Auto-scale	Yes (with HPA)	Not typically	Rare, often managed manually
Pod ordering	No	No	Yes (start/stop order)

* Use Deployment for scalable, stateless applications like frontend apps or APIs.

* Use DaemonSet for background node-level agents like monitoring, log collectors.

* Use StatefulSet for stateful apps like databases (e.g., MySQL, Cassandra) that need stable pod identities, ordered deployments, and persistent volumes. StatefulSets retain volume identity even after rescheduling.

21) Let us suppose you have to setup Dynatrace agent for monitoring on every node in your eks cluster. How will you plan the setup? OR How do you deploy Dynatrace agent on a cluster for monitoring?

Answer:

Need one pod per node = daemonset.

Use the Dynatrace Operator or Helm chart to deploy the agent. It typically sets up a **DemonSet** so that each node runs a monitoring pod. This pod collects metrics like CPU, memory, network, and sends them to the Dynatrace backend. Ensure proper permissions, namespace setup, and connect to your Dynatrace environment.

57	
58	
59	
60	
61	
62	
63	
64	

IMP 22) What are different deployment strategy available in kubernetes?

Answer:

Strategy	What happens	Pros	Cons	Use Case	Example YAML/Setup
Rollout	Deletes all old pods, then creates all new pods.	Simpler to manage between versions.	Downtime during switch.	Apps where downtime is acceptable (e.g. batch, non-critical services).	<code>kind: Deployment</code> (or <code>Rollout</code>)
Rolling Update (default)	Gradually replaces old pods with new pods while keeping same number.	No downtime if done correctly.	Rollback is slower if issue occurs mid-way.	Most web services, APIs, and standard apps.	<code>kind: Deployment</code> (or <code>Rollout</code>)
Blue-green	Two environments, "blue" (current) and "green" (new), both mirrored when ready.	Instant rollback possible, zero downtime.	Double resource usage temporarily.	Zero-downtime releases with quick rollback needs.	Two Deployments + Service selector change.
Canary	Release new version to small subset first, then increase gradually.	Limits risk, allows real-world testing.	Needs traffic splitting setup.	Testing risky changes with live traffic.	Multiple Deployments + weighted routing via Istio/Ingress.

23) Which component of Kubernetes is responsible for maintaining your desired configuration?

Answer:

The kube-controller-manager watches the desired state (as defined in manifests) and ensures the actual state [see from etcd] matches it. For example, it maintains the number of pod replicas, node health, endpoints, and handles jobs. It works alongside the scheduler and API server to ensure Kubernetes is always in the expected condition.

Different kinds of controllers

- ReplicationController / ReplicaSet Controller – Ensures the desired number of pod replicas are running.
- Deployment Controller – Manages rolling updates, rollbacks, and scaling of deployments.
- StatefulSet Controller – Manages stateful pods with stable IDs and storage.
- DaemonSet Controller – Ensures pods run on all or specific nodes.

58

59

60

61

62

63

64

65

Job Controller – Runs pods until a task completes.

CronJob Controller – Schedules jobs to run at specific times.

Node Controller – Monitors node health and handles node failures.

Namespace Controller – Cleans up resources when a namespace is deleted.

ServiceAccount & Token Controller – Manages service accounts and access tokens.

EndpointSlice / Service Controller – Keeps service endpoints updated.

HorizontalPodAutoscaler (HPA) Controller – Scales pods based on resource usage.

24) Explain what happens when your pod wants to list down secrets in the same namespace? Complete flow of request.

Answer:

Flow of request:

1. **Create a ServiceAccount** (e.g., my-sa).
2. **Create a Role** that gives permission to **list secrets** in the namespace.
3. **Create a RoleBinding** that binds the Role to the ServiceAccount.
4. In the **pod spec**, set serviceAccountName: my-sa.
5. When the pod tries to list secrets, **Kubernetes API Server** checks:
"Is my-sa allowed to list secrets in this namespace?"
6. ☒ If yes, access is granted. ☐ If not, returns 403 Forbidden.

Here's how RBAC works in Kubernetes when a pod tries to do something:

Example: A pod wants to list secrets

1. You create a `ServiceAccount`:
2. You create a `Role`, with permission to list secrets:
3. You create a `RoleBinding` to bind `my-sa` with that Role
4. You assign `my-sa` in the pod's `serviceAccountName`:

Now, when the pod tries to `list` secrets, the Kubernetes API Server checks:

Is this service account (`my-sa`) allowed to list secrets in this namespace?

- If yes: access allowed
- If no: 403 Forbidden

RBAC actually has components role and rolebinding which are used to give permission to a serviceaccount. That is how RBAC works for pods. 4 concepts in RBAC:

• **Role YAML (to allow listing secrets)**

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: secret-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "list"]
```

• **RoleBinding YAML**

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: secret-reader-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-sa
  namespace: default
roleRef:
  kind: Role
  name: secret-reader
  apiGroup: rbac.authorization.k8s.io/v1
```

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

60

61

62

63

64

65

66

67

Pod Spec (Partial)

```
apiVersion: v1
kind: Pod
metadata:
  name: secret[[ist-pod]]
spec:
  serviceAccountName: my-sa
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "kubectl get secrets"]
```

```
(ec2-user@ip-172-31-9-150 ~)$ kubectl describe pod/flask-deployment-857d4c8b7-f297d
Name:         flask-deployment-857d4c8b7-f297d
Namespace:    default
Priority:      0
ServiceAccount: default
Node:         minikube/192.168.49.2
Start Time:   Tue, 09 Sep 2020 15:22:55 +0000
Labels:       app=flask
              pod-template-hash=857d4c8b7
Annotations:  none
Status:       Running
IP:           10.244.0.101
IPs:          10.244.0.101
Controlled By: ReplicaSet/flask-deployment-857d4c8b7
Containers:
  flask-container:
    Container ID:   docker://209f644b9f2b44d6d4b3d4d7089e74927521aef2176c4e0062829376e
    Image:          flask-app:1.0.1
    Image ID:       docker://sha256:8ec67ccae077f4b64427972b0e6cc0695d9399f1e209ccac0
    Port:           5000/TCP
    Host Port:      0/TCP
    State:          Running
    Started:        Tue, 09 Sep 2020 15:22:55 +0000
```

25) Does a pod run RBAC logic inside itself?

Answer:

No — RBAC is enforced by the API server, not by the pod.

- RBAC doesn't directly run inside a pod but pods running a service account
- When a pod makes API calls to Kubernetes (e.g., checking config, listing secrets) Kubernetes checks the service account assigned to that pod, and applies RBAC rules to authorize or deny the request

Example: A pod using `default` wants to list pods → API Server checks if `default` is bound to a Role that allows listing pods.

But, the identity of the pod (ServiceAccount) is what the API server uses to decide whether to allow or deny requests. RBAC is a concept based on 4 pillars:

Role, rolebinding, clusterrole, clusterrolebinding.

RBAC actually has components role and rolebinding which are used to give permission to a serviceaccount. That is how RBAC works for pods. We create a SA. We create a role. We assign that role to SA via rolebinding. Now pod places request and apiserver verifies to which SA is associated to this pod and checks what role this SA has. And accordingly, denies/allows the action.

26) How does pod get a service account?

2. How does a pod get a ServiceAccount?

Every pod in Kubernetes runs using a ServiceAccount, which acts as its identity.

- If you don't specify anything, Kubernetes assigns the default service account of the namespace.

```
apiVersion: v1
kind: Pod
spec:
  serviceAccountName: default
```

- If you want custom permissions, you create and assign a custom ServiceAccount:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  serviceAccountName: my-sa
  containers:
  - name: app
    image: myimage:v1
```

This `my-sa` must be bound to a Role or ClusterRole to actually have permissions!

27) What k8 components are involved in RBAC?

Answer:

- **API Server:** Core component. It checks the RBAC rules every time a request is made.
- **etcd:** Stores RBAC objects (Role, Binding etc).
- **Controller Manager:** Not directly involved in checking RBAC, but uses service accounts internally.
- **Scheduler:** Not affected by RBAC directly.

28) What happens when you create a Service of type "load balancer" in kubernetes cluster?

Answer:

- You apply a Kubernetes Service of type LoadBalancer.
- Kubernetes automatically assigns a **nodePort** to the service (e.g., 31677).
- The service also selects pods using the provided selector (e.g., app=ingress-nginx).
- The Kubernetes Cloud Controller detects the LoadBalancer type service.
- It triggers AWS API to create a Network Load Balancer (NLB).
- AWS creates the Load Balancer and attaches all EKS worker nodes as targets on nodePort.
- The Load Balancer is assigned a public DNS name (e.g., abc123.elb.amazonaws.com).
- On each worker node, kube-proxy sees the service and its NodePort.
- kube-proxy programs iptables (or IPVS) rules to forward traffic from NodePort to pods.
- When an external user hits the LB DNS (port 80), traffic reaches a node on the NodePort (e.g., node-ip:31677).
- The node's kube-proxy forwards the request to one of the backend pods (e.g., ingress-nginx).
- The ingress-nginx pod receives the request and applies the routing logic defined in the Ingress resource.
- The request is routed internally to the correct service and pod (e.g., /login → login-service → login-pod).

Step-by-Step: What Happens When You Create a Service of Type LoadBalancer

- Step 1: You apply a Service YAML

```
apiVersion: v1
kind: Service
metadata:
  name: my-ingress-nginx
spec:
  type: LoadBalancer
  ports:
  - port: 80
    targetPort: 80
  selector:
    app: ingress-nginx
```

You're saying "Expose this group of pods (ingress-nginx) to the external world via a cloud Load Balancer."

- Step 2: Kubernetes assigns a NodePort

- Even though you didn't specify it, Kubernetes automatically assigns a nodePort, like 31677.
- Now this service behaves like a NodePort service, too.

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Share

64

65

66

67

68

69

70

71

When you apply this:

Kubernetes adds:

```
port
nodePort: 31677 # This port added automatically by Kubernetes
```

So every LoadBalancer service is also a NodePort service internally.

You can see this by running:

```
kubectl get svc ingress-nginx-controller -o yaml
```

It will show:

```
port:
  - port: 80
    targetPort: 80
    nodePort: 31677
```

The same Service: LoadBalancer handles everything:

AWS LB points to node-ip:nodePort (e.g., 31677)

65

66

67

68

69

70

71

72

- **Step 3: Kubernetes Cloud Controller triggers AWS API**
 - The Cloud Controller Manager (in EKS: AWS-specific controller) sees `type: LoadBalancer`.
 - It makes an API call to AWS:
 - Creates an NLB (Network Load Balancer) or CLB/ALB based on annotation
 - Adds all worker nodes as targets
 - Sets target group to forward to `node-ip:nodePort`
- **Step 4: AWS Creates the Load Balancer**
 - The Load Balancer gets a public DNS (e.g., `a-b-c-elb.amazonaws.com`)
 - The Load Balancer's **target group** is configured:
 - Protocol: TCP
 - Targets: All EC2 worker nodes (on `nodePort`)
- **Step 5: kube-proxy Programs Routing Rules**
 - On each node, `kube-proxy` sees:
 - A service exists on port `31677`
 - That service selects pods with label `app=ingress-nginx`
 - It creates `iptables/IPVS` rules:
 - Any traffic to `node-ip:31677` → forward to one of the matching pod IPs

66

67

68

69

70

71

72

- Step 6: You get a Public Endpoint
 - The Load Balancer DNS is now publicly reachable
 - When someone hits `http://xlb-dns:80`, here's what happens:

```
Client → AWS Load Balancer → node-ip:31677 → kube-proxy → ingress-nginx pod (port 80)
```
- Step 7: Ingress Controller Takes Over
 - The ingress-nginx pod receives traffic
 - It reads the Ingress rules
 - Forwards traffic to the actual backend services and pods (e.g., login, product, etc.)
- Bonus: Internals of the Service

You can see the assigned NodePort by running:

```
kubectl get svc my-ingress-nginx -o yaml
```

It'll look like:

```
ports:
- port: 80
  targetPort: 80
  nodePort: 31677
```

29. BackoffLimit in k8 jobs?

73

```
apiVersion: batch/v1
kind: Job
metadata:
  name: my-job
spec:
  template:
    spec:
      containers:
        - name: python
          image: python
          command: ["echo", "hello", "world"]
          restartPolicy: Never
      backoffLimit: 3
```

Answer:

If a pod inside a Job fails (exit code $\neq 0$), Kubernetes will restart it.

backoffLimit defines how many retries are allowed before the whole Job is marked as failed.

Once retries exceed the backoffLimit, the job status changes to Failed.

The default value of backoffLimit is 6.

30) What happens internally when we run `kubectl apply -f deployment.yaml`?

1. kubectl reads the manifest file and sends it to the API server.
2. The API server authenticates and validates the request.
3. If valid, it stores the desired state in etcd.
4. The scheduler picks a node and assigns the pod.
5. The kubelet on the node pulls the image and runs the container.
6. The pod becomes "Running" once all containers are ready.

Real-world: Like giving an order to a waiter. kubectl = customer, API server = waiter, etcd = kitchen instructions, kubelet = cook.

31) What is the role of kube-proxy in Kubernetes?

1. kube-proxy runs on every node.
2. It manages networking rules (iptables or IPVS) to route traffic to pods.
3. It ensures service IP (ClusterIP) maps to backend pod IPs.
4. Supports round-robin load balancing between pod endpoints.

Real-world: Like a traffic cop redirecting cars to available lanes (pods).

32) What are Kubernetes CRDs (Custom Resource Definitions)?

1. CRDs let you define your own Kubernetes object types.

2. You can manage them like built-in resources (e.g., Pods, Deployments).

3. Useful for extending Kubernetes without modifying the source code.

4. Tools like ArgoCD and Istio use CRDs heavily.

```
apiVersion: "example.com/v1"
kind: MyApp
metadata:
  name: sample
spec:
  replicas: 2
```

33) What is a Headless Service in Kubernetes and when do you use it?

1. A **Headless Service** is a Kubernetes service without a ClusterIP (clusterIP: None).

2. It doesn't load balance — instead, it **returns a list of pod IPs** directly via DNS.

3. Commonly used for **StatefulSets**, **database clusters**, or direct pod-to-pod communication.

4. It enables **service discovery** where the client can choose which pod to talk to.

```
apiVersion: v1
kind: Service
metadata:
  name: my-db
spec:
  clusterIP: None
  selector:
    app: my-db
  ports:
    - port: 5432
```

When you query DNS:

```
nslookup my-db.default.svc.cluster.local
```

→ It returns **all pod IPs**, not a single VIP.

34) Why do we use a Headless Service with a StatefulSet in Kubernetes instead of a normal ClusterIP Service?

Answer: **ClusterIP/NodePort/LB use load balancing** — they expose **one virtual IP (VIP)**, and traffic is randomly distributed to backend pods.

69

70

71

72

73

74

75

76

In some cases (like databases, caching systems, etc.), **each pod must be uniquely reachable**. Load balancing breaks this requirement.

Headless Service disables load balancing (clusterIP: None) and exposes individual pod DNS entries, like:

```
my-db-0.my-db.default.svc.cluster.local
my-db-1.my-db.default.svc.cluster.local
```

It is **essential for StatefulSets**, where:

- Pod identity matters (my-app-0, my-app-1)
- Each pod might have **different data or responsibilities**.
- StatefulSet pods need stable identities (like mongo-0, mongo-1) across restarts and reschedules.
- A headless service (clusterIP: None) gives each pod a stable DNS name using predictable naming:
`<pod-name>.<service-name>.<namespace>.svc.cluster.local`

This enables **direct communication between specific pods**, which is required for:

- DB replication (e.g., mongo-1 replicating from mongo-0)
- Leader election (e.g., Kafka, RabbitMQ)
- Stateful data syncing between nodes

A regular ClusterIP or LoadBalancer **hides individual pod IPs** behind a single VIP — not suitable when specific pods must talk to each other.

35) By default, how does a Kubernetes Service forward traffic to backend pods? How does load balancing work in ClusterIP, NodePort, and LoadBalancer?

- A Kubernetes **Service creates a virtual IP (ClusterIP)**.
- kube-proxy (running on each node) sets up **iptables or IPVS rules**.
- These rules **redirect traffic from the Service IP to one of the matching pod IPs**. (load balancing)
- The selection is usually **round-robin** (default behaviour).

70

71

72

73

74

75

76

77

36) What are annotations in K8 ?

Answer: In Kubernetes, annotations are key-value pairs attached to objects (like Pods, Deployments, Services, etc.) that store metadata which is not meant to identify or select the object, but to provide extra information.

first example is for extra info on pod level and second yaml is for using sticky session on Ingress.

"In Kubernetes, **Ingress controllers** (like NGINX Ingress) support **sticky sessions** (session affinity), which ensure that all requests from the same client go to the same backend pod. This is often required for applications that maintain state in memory (like login sessions, shopping carts, etc.)"

Key points

- 1. **Difference from Labels**
 - Labels used for selection (the `selector` in pods, etc.)
 - Annotations used for storing **information**, not for selection.
- 2. **Formats of yaml**
 - Adding label or response into GET request, make test.
 - Scaling control (think of the user seeing a pod).
 - Configuring external tools (Ingress controllers, monitoring tools, service meshes).
 - Attaching links to operators or administrator consoles.
- 3. **Types in Yaml**

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    description: "This pod is used for testing"
    job: "backend"
    owner: "John Doe"
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    ingress.kubernetes.io/rewrite-target: "/"
    ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
  - host: "example.com"
    http:
      paths:
      - path: "/"
        pathType: Prefix
        backend:
          service:
            name: "example-service"
            port:
              number: 80
```

Batch_17_DevOps_Master_Interview_question_Set_Advanced.pdf

File View Tools Help

Page 77 / 158

75%

71

72

73

74

75

76

77

78

Answer: In Kubernetes,

1) Liveness Probe

- **Purpose:** Tells Kubernetes whether a container is still healthy (alive).
- **Action on failure:** Kubernetes restarts the container if the probe fails repeatedly.
- Used to detect deadlocks, hung processes, or unrecoverable failures while the container is still running.
- **Example:** if the app returns non-200/300 statuses too often or becomes unresponsive repeatedly, Kubernetes restarts it.

2) Readiness Probe

- **Purpose:** Tells Kubernetes whether a container is ready to serve traffic.
- **Action on failure:** The pod is **taken out of service** (removed from endpoints) but the container is **not restarted**.
- Used when the app is temporarily unable to serve — e.g., slow processing, waiting on a dependency, or during intermittent load spikes.
- This prevents traffic being sent to a slow/unready pod while preserving in-progress work.

3) Startup Probe

- **Purpose:** Used for slow-starting applications to delay other probes until the app is fully started.
- **Action before success:** While the startup probe is in progress, **liveness and readiness probes are disabled**.
- Once the startup probe succeeds, normal liveness/readiness checks take over.
- Prevents premature restarts during long initialization phases instead of using large initial delays on other probes.

72

73

74

75

76

77

78

79

How They Behave in the Lifecycle

- On pod start: **Startup probe runs first** (if configured).
 - Until it succeeds, Kubernetes **doesn't** run liveness or readiness probes.
- After startup succeeds:
 - Liveness** checks continue periodically to ensure the container hasn't gone unhealthy — on failure → **restart**.
 - Readiness** checks continue to decide if the pod should receive traffic — on failure → **remove from load balancer / services**, but **don't restart**.

Key Differences

Probe type	What it Detects	Action on Failure
Liveness	If app is alive/healthy	Restart container
Readiness	If app is ready to receive traffic	Remove from service endpoints (no restart)
Startup	Whether app finished initial startup	Delays other probes until success

Why Use Them

- Liveness**: recover from internal deadlocks without waiting for crash.
- Readiness**: prevent traffic to temporary slow/unready pods without killing them.
- Startup**: avoid false failure detections on slow-starting containers.

Example: Pod with all three probes

```
apiVersion: v1
kind: Pod
metadata:
  name: probe-demo
spec:
  containers:
  - name: app
    image: myapp:v1.0
    ports:
    - containerPort: 8080

    # Startup Probe
    startupProbe:
      httpGet:
        path: /health/startup
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
      # App can take up to 5 minutes (30 * 10s) to start
      # Until this succeeds, liveness & readiness are disabled

    # Readiness Probe
    readinessProbe:
      httpGet:
        path: /health/ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
      failureThreshold: 3
      # If this fails + pod removed from service endpoints
      # Container is NOT restarted

    # Liveness Probe
    livenessProbe:
      httpGet:
        path: /health/live
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 10
      failureThreshold: 3
      # If this fails repeatedly + container is restarted
```

38) What are PDB PodDisruptionBudget in K8 ?

Answer: In Kubernetes,

PodDisruptionBudget ensures minimum availability of pods during planned disruptions like node drain, preventing too many pods from being evicted at once. Two ways to define it (use only one):

1. minAvailable
2. maxUnavailable

minAvailable: 3 means At least 3 pods must always be running

maxUnavailable: 1 means At most 1 pod can be down at any time

How Kubernetes enforces PDB: When eviction is requested:

1. K8s checks the PDB
2. If budget allows → pod is evicted
3. If budget exceeded → eviction is **blocked**
4. Node drain may **pause** until budget allows

40) What are abc in K8 ?

Answer: In Kubernetes,

TERRAFORM Interview Questions & Answers

1) How do you manage the state file in Terraform?

Answer:

- ☑ Use **remote backends** like **S3 + DynamoDB** for storing the state file and state locking.
- ☑ Enable **versioning** in S3 to recover old state if needed.
- ☑ Avoid storing state locally to prevent conflicts in team environments.



Search



ENG
IN

12:16
17-05-2026